

Module 5 Week A — Applied Lab: Regression & Evaluation

In this lab you will build a complete train-evaluate loop in scikit-learn using the Petra Telecom customer churn dataset. By the end, you will have trained logistic regression and linear regression models with regularization, evaluated them with standard metrics, and run cross-validation – all inside scikit-learn Pipelines.

Due: Tuesday end of day. Resubmissions are accepted through Saturday of the assignment week.

Learning Objectives

- Split data into train and test sets with stratification
- Build scikit-learn Pipelines with `StandardScaler` and a model
- Evaluate a classifier with accuracy, precision, recall, F1, and a confusion matrix
- Evaluate a regressor with MAE and R-squared
- Compare Ridge and Lasso regularization by inspecting coefficients
- Run stratified k-fold cross-validation and interpret the results

Note

The classification task in this lab uses the **same** `telecom_churn.csv` dataset and the **same** `class_weight="balanced"` **Logistic Regression configuration** as the Module 5 Week A drill. It's the same model on the same imbalanced data (~16% churn).

This same dataset and model configuration recur in Thursday's Integration Task which builds an explicit `DummyClassifier` baseline comparison to tie the whole thing together. Section 5 of the Week A reading already explained why accuracy is misleading on imbalanced data such as this. For this lab, apply your mechanical skills on the split/pipeline/metrics/CV workflow and pay close attention to Task 7's reflective questions. The regression tasks (Ridge/Lasso on `monthly_charges`) are separate from this issue — `monthly_charges` is a simple continuous target with no class imbalance, so your R² and MAE numbers will directly show how well your model fits the data.

Setup

1. **Accept the GitHub Classroom assignment:** [Accept the Lab 5A assignment](#)
2. **Clone your repo:**

```
git clone <your-repo-url>
cd <your-repo-name>
```

3. **Install dependencies:**

```
pip install -r requirements.txt
```

4. **Create a working branch:**

```
git checkout -b lab-5a-regression-eval
```

The starter file `lab_regression.py` contains function signatures with docstrings and TODO comments. The dataset is at `data/telecom_churn.csv`. Read the TODO directions, consult the Week A reading, and implement each function.

Task 1: Load Data and Basic EDA

Load `data/telecom_churn.csv` into a pandas `DataFrame`. Print the shape, check for missing values, and examine the distribution of the target column `churned`. This is a brief check – you covered thorough EDA in Module 4. Here, confirm the data is clean enough to model.

Task 2: Split the Data

Use `train_test_split` to create an 80/20 train/test split. Set `random_state=42` and `stratify` on the `churned` column. Print the sizes of each split and the churn rate in both to confirm stratification worked.

Task 3: Logistic Regression Pipeline

Build a `Pipeline` with two steps:

1. `StandardScaler` to normalize features
2. `LogisticRegression` with `random_state=42`, `max_iter=1000`, and `class_weight="balanced"`

Fit the pipeline on the training data. Generate predictions on the test set. Print the full `classification_report` and display the confusion matrix using `ConfusionMatrixDisplay`. Your function must also return a dictionary with keys `"accuracy"`, `"precision"`, `"recall"`, `"f1"` – the autograder checks this return value.

Task 4: Ridge Regression Pipeline

Switch targets: use `monthly_charges` as the regression target (you will need to re-split with `monthly_charges` as `y`). Note that `monthly_charges` is a continuous value – you cannot use `stratify` on a continuous target. Your `split_data` function should handle this case. Build a `Pipeline` with `StandardScaler` and `Ridge(alpha=1.0)`. Fit on training data, predict on the test set, and report MAE and R-squared.

Task 5: Lasso Regularization Comparison

This task is not autograder-tested. Write your code using the `build_lasso_pipeline()` stub in the starter file.

Build a second regression pipeline using `Lasso(alpha=0.1)` on the same `monthly_charges` target. After fitting, print the coefficient for each feature from both the Ridge and Lasso models side by side. Identify which features Lasso drives to zero. Briefly note in a code comment why these features may be uninformative for predicting monthly charges.

Task 6: Cross-Validation

Return to the classification target (`churned`). Run 5-fold stratified cross-validation on your logistic regression pipeline using `cross_val_score`. To make results reproducible across the cohort, construct the CV splitter explicitly and pass it via the `cv=` parameter:

```
cv_splitter = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(pipeline, X_train, y_train, cv=cv_splitter, scoring="acc
```

Print the score for each fold and the overall mean +/- standard deviation. Using the explicit splitter (rather than `cv=5`) ensures everyone sees the same CV numbers.

Task 7: Summary of Findings

This task is not autograder-tested. Add your summary directly in your script.

Add a summary as a docstring or comments at the bottom of your script answering:

- Which features appear most important for predicting churn?
- How does the logistic regression model perform? Is precision or recall more concerning for this problem?
- What would you recommend as next steps to improve performance?

Challenge Extensions

These are optional extensions for learners who want to go deeper. They are not graded and do not affect your lab score. No starter code is provided – design and implement these independently.

Tier 1: Threshold Tuning

The default classification threshold is 0.5. For churn prediction, missing a churner (false negative) may be more costly than a false alarm (false positive).

- Use `predict_proba` to get probability scores for the test set.
- Evaluate precision, recall, and F1 at thresholds of 0.3, 0.4, 0.5, 0.6, and 0.7.
- Plot precision and recall as a function of threshold.
- Identify the threshold that maximizes F1 and explain why it differs from 0.5 for this dataset.

Tier 2: Config-Driven Model Sweep

Replace hard-coded model configurations with a YAML or JSON config file that defines a list of models and their hyperparameters. Write a function that reads the config, instantiates each model inside a `Pipeline`, runs cross-validation, and produces a formatted results table. The function should work for any config file – not just the models used in this lab. Test with at least 6 configurations (vary C, alpha, penalty type, and solver).

Tier 3: Implement Logistic Regression from Scratch

Implement binary logistic regression using only NumPy (no scikit-learn for the model itself). Your implementation must include:

- Sigmoid function
- Log-loss (binary cross-entropy) cost function
- Gradient descent optimization with a configurable learning rate and iteration count
- L2 regularization as an additive penalty in the gradient
- A `fit(X, y)` and `predict(X)` interface

Compare your implementation's predictions and coefficients to scikit-learn's `LogisticRegression` on the same data. Document where results diverge and why (solver differences, convergence criteria, default regularization).

Submission

Open a pull request from `lab-5a-regression-eval` to `main`. Your PR description must include:

1. Classification report for logistic regression (copy-paste the text output)
2. R-squared and MAE for Ridge regression
3. Brief summary of findings (3–5 sentences)
4. Paste your PR URL into TalentLMS → Module 5 Week A → Lab 5A to submit this assignment.

